TESTNG CLASS 7

Excel Data-Driven Testing

Apache POI • TestNG DataProvider • Selenium WebDriver

Beginner-to-practical guide with commented code, real testing scenarios, troubleshooting, and best practices

READ EXCEL	SUPPLY DATA	RUN TEST CASES

What you will learn

How to store test inputs in an Excel workbook, read them safely with Apache POI, pass each row to a TestNG @Test method using @DataProvider, and run Selenium checks repeatedly without duplicating test code.

Prepared from the supplied "TestNG class 7" notes, expanded with practical corrections and framework guidance.

Contents and learning path

1. Why data-driven testing is useful
2. Excel files and Apache POI basics
3. Reading a single cell from Excel
4. Cell types, formatted values, phone numbers, and common errors
5. TestNG @DataProvider and Object[][]
6. Moving from hard-coded data to Excel-backed data
7. A reusable ExcelDataReader implementation
8. End-to-end Selenium + TestNG example
9. Project structure and Maven configuration
10. Troubleshooting, best practices, exercises, answers, and interview revision

Source note

Your class material demonstrates FileInputStream, XSSFWorkbook, XSSFSheet, row/cell indexing, a hard-coded Object[][] DataProvider, an Excel-driven DataProvider, and a reusable nested-loop approach. Those topics form the core of this guide. Additional safeguards are clearly presented as practical improvements.

1. What is data-driven testing?

Data-driven testing means running the same test logic with different input data. Instead of writing separate methods for every user, product, or account, the inputs are stored separately and supplied to one reusable test method.

Simple example

Test data row	First name	Last name	Phone
1	Amol	Jadhav	9090901001
2	Rutuja	Kale	9090901002
3	Sonali	Jadhav	9090901003
4	Vikas	Mule	9090901004

The registration test can run four times: once for each row. The code that locates fields and enters values stays the same; only the row values change.

Why teams use it

- Reduces duplicate test code and makes tests easier to maintain.
- Makes it easy to add positive, negative, boundary, and role-based test data.
- Allows testers to update approved test inputs without editing Java code.
- Improves coverage when each data row represents a distinct scenario.
- Keeps the test logic separate from the test data, a useful design principle for UI and API automation.

Typical enterprise example

A login suite may test a valid user, an invalid password, a locked account, a blank username, and a blank password. Each scenario can be one data row with expected outcome fields. The test must assert the expected result, not merely fill the form.

Important distinction

Data-driven testing is a technique, not a guarantee of test quality. Poor or duplicate data still produces poor coverage. Define what each row is proving and what result should pass or fail.

2. Apache POI and Excel file basics

Apache POI is a Java library for reading and writing Microsoft Office files. Your notes use the XSSF API for modern .xlsx workbooks.

Excel file type	Typical POI API	Notes
.xls (older Excel format)	HSSFWorkbook / HSSF	Legacy binary Excel workbook support.
.xlsx (modern Excel format)	XSSFWorkbook / XSSF	Office Open XML workbook support.
Either format (when supported)	WorkbookFactory	Can choose an appropriate Workbook implementation based on the file contents.

Main objects

Class / interface	Purpose
FileInputStream or InputStream	Opens the Excel file bytes from disk.
Workbook / XSSFWorkbook	Represents the Excel workbook file.
Sheet / XSSFSheet	Represents one worksheet tab, such as Register.
Row / XSSFRow	Represents one row in the worksheet.
Cell / XSSFCell	Represents a single cell inside a row.
DataFormatter	Converts a cell to a string resembling the displayed Excel value.
FormulaEvaluator	Calculates formula cells when formatted values are needed.

Maven dependency

Add Apache POI to pom.xml. The artifact used for XSSF and the common spreadsheet APIs is poi-ooxml. Check the Java version in your project before choosing the POI major version.

Example dependency for a POI 5.x project

```
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi-ooxml</artifactId>
  <version>5.5.1</version>
</dependency>
```

Version note: Apache POI 6.0.0 was announced as the latest stable release in October 2026 and requires Java 17 or newer. The 5.5.1 example is shown for projects that need to remain on the POI 5.x line. Use a version compatible with your JDK and your organization's dependency/security policy.

Suggested project layout

```
my-automation-project/
 pom.xml
 src/
  test/
   java/
    data/ExcelDataReader.java
    providers/RegisterDataProvider.java
    tests/RegisterTest.java
   resources/
    testdata/Register.xlsx
```

Keeping Excel under src/test/resources is more portable than referencing a developer-specific path such as C:\Users\someone\Desktop\TestData.xlsx. In larger Maven projects, you may instead load test data from the classpath or a configurable external test-data directory.

3. Read a single value from an Excel sheet

Your original code follows four steps: open a file stream, create a workbook, select a sheet, then select a row and cell. Excel row and column indexes in Apache POI are zero-based.

Example: read the value in row 0, column 0

```
import java.io.FileInputStream;
import java.io.IOException;
import org.apache.poi.xssf.usermodel.XSSFSheet;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;

public class ReadExcelDemo {
   public static void main(String[] args) throws IOException {
      String filePath = "src/test/resources/testdata/Register.xlsx";

      try (FileInputStream fis = new FileInputStream(filePath);
           XSSFWorkbook workbook = new XSSFWorkbook(fis)) {

         XSSFSheet sheet = workbook.getSheet("Register");

         String firstName = sheet.getRow(0)
                                 .getCell(0)
                                 .getStringCellValue();
         System.out.println(firstName);
      }
   }
}
```

Line-by-line explanation

11. FileInputStream opens the workbook file so POI can read its bytes.
12. XSSFWorkbook parses the .xlsx workbook. Use try-with-resources so the stream and workbook are closed reliably.
13. getSheet("Register") selects the tab named Register. The sheet name must match the workbook exactly.
14. getRow(0) selects the first row; getCell(0) selects the first column (column A).
15. getStringCellValue() works when the target cell stores a string. It can fail if the cell is numeric, a formula, or another type.
16. System.out.println prints the extracted value to the console.

Use this for learning, not as the final reusable reader

Direct chained calls are concise but can throw a NullPointerException when the sheet, row, or cell is missing. Production-grade data readers should validate these objects and produce an error message that identifies the sheet and cell.

4. Excel row/column indexes and cell values

Excel position	Apache POI index	Meaning
A1	row 0, cell 0	First row, first column.
C1	row 0, cell 2	First row, third column.
B3	row 2, cell 1	Third row, second column.
F4	row 3, cell 5	Fourth row, sixth column.

Methods seen in the source notes

Method	When to use / risk
getStringCellValue()	Use only when the cell is known to contain text. A numeric phone cell may throw an exception.
getRawValue()	Specific to XSSF cell implementations; exposes raw stored value and is not the same as the displayed Excel text. Avoid relying on it for generic cell handling.
DataFormatter.formatCellValue(cell)	Returns a display-oriented text value and handles different cell types more flexibly.
DataFormatter.formatCellValue(cell, evaluator)	Can format formula results using a FormulaEvaluator.

Phone numbers: keep them as text

Phone numbers are identifiers, not quantities for arithmetic. If you store a phone number as a numeric cell, leading zeros can be lost and formatting can become surprising. Format the Excel phone column as Text and store the intended digits as text. In Java, pass it as String.

Example

If a phone number is written as 0091-080-001234 in Excel, storing it as numeric data may remove leading zeros or formatting. A text cell preserves the representation the test should enter.

Common Excel problems

- Sheet name mismatch: getSheet("Register") returns null if there is no exact matching tab.
- Blank row or cell: getRow() or getCell() may return null.
- Numeric data: getStringCellValue() may throw an IllegalStateException.
- Formula cell: use a FormulaEvaluator if the value must reflect the calculated result.

- Date cells: the visible text can be locale/formatted; agree on a stable data format such as yyyy-MM-dd or store the test value as text.
- Header row: do not accidentally execute the header values as one test invocation. Skip it deliberately.

5. TestNG @DataProvider fundamentals

@DataProvider supplies rows of arguments to a TestNG test method. It is commonly used for data-driven testing.

Simple example without Excel

```
@DataProvider(name = "registerTestData")
public Object[][] getTestData() {
   return new Object[][] {
      { "Amol", "Jadhav" },
      { "Rutuja", "Kale" }
   };
}

@Test(dataProvider = "registerTestData")
public void registerTestCase(String firstName, String lastName) {
   System.out.println(firstName + " " + lastName);
}
```

How Object[][] works

Expression	Meaning
Object[][]	A two-dimensional array of objects.
Outer array / rows	Each outer row supplies one invocation of the test method.
Inner array / columns	Each value becomes one argument for that invocation.
2 rows x 2 values	The test method runs twice, with two arguments each time.

The DataProvider name and dataProvider attribute must match. The number and types of values in each data row must match the test method's parameters.

Correction to a common misunderstanding

If each data row contains four values, the test method normally needs four matching parameters, not five. TestNG maps the supplied values to the method parameters in order.

Four rows and three columns

```
Object[][] data = new Object[4][3];
data[0][0] = "Amol";
data[0][1] = "Jadhav";
data[0][2] = "9090901001";
// Continue filling rows 1, 2 and 3...
return data;
```

This shape can supply four test invocations; each invocation receives three arguments. In normal code, use a literal array or populate it in a loop instead of manually repeating many assignments.

6. DataProvider with hard-coded data

```
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

public class RegisterDataProviderDemo {
   @DataProvider(name = "registerTestData")
   public Object[][] getTestData() {
      return new Object[][] {
         { "Amol",   "Jadhav", "9090901001" },
         { "Rutuja", "Kale",   "9090901002" },
         { "Sonali", "Jadhav", "9090901003" },
         { "Vikas",  "Mule",   "9090901004" }
      };
   }

   @Test(dataProvider = "registerTestData")
   public void registerTestCase(String firstName,
                                String lastName,
                                String phone) {
      System.out.println(firstName + " " + lastName + " " + phone);
   }
}
```

Explanation

- @DataProvider(name = "registerTestData") gives a stable name to the data source.
- Object[][] contains four rows of input and three columns per row.
- @Test(dataProvider = "registerTestData") tells TestNG which provider to call.
- Each row is passed to registerTestCase as firstName, lastName, and phone.
- The test runs four times; it does not mean there are four separate Java test methods.

When hard-coded test data is acceptable

A small fixed data set is useful for a unit test, a quick demonstration, or a very stable few-case test. For larger or frequently edited datasets, move data to a separate source such as Excel, CSV, JSON, or a database.

7. Read Excel data and return Object[][]

Your source material first reads four Excel rows manually and then introduces a reusable nested-loop method. The reusable approach is better because adding more rows does not require adding more assignments in Java.

The nested-loop idea

Basic pattern shown in the original notes

```
int totalRows = sheet.getLastRowNum() + 1;
int totalCells = sheet.getRow(0).getLastCellNum();
Object[][] data = new Object[totalRows][totalCells];

for (int i = 0; i < totalRows; i++) {
   for (int j = 0; j < totalCells; j++) {
      data[i][j] = sheet.getRow(i).getCell(j).getStringCellValue();
   }
}
return data;
```

The outer loop walks row-by-row. The inner loop walks through the cells in the current row. Each cell value is copied into the matching position in the Java Object[][] array.

Why production code needs extra checks

- Row indexes are zero-based; getLastRowNum() is an index, not a count of populated rows. Gaps and formatted blank rows may exist.
- Rows and cells can be missing, getLastCellNum() can vary by row, and getStringCellValue() assumes text.
- Skip the header, ignore fully blank rows, validate expected columns, and close the workbook/stream even after an error.

8. Reusable ExcelDataReader: safer implementation

The following example keeps the source lesson's purpose—read all cells into Object[][]—but adds practical validation, safe closing, blank-cell handling, formula evaluation, and header-row skipping. It formats values to strings so the matching test method can accept String parameters.

Reusable helper class

```
package data;
import java.io.IOException;
import java.io.InputStream;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.List;
import org.apache.poi.ss.usermodel.Cell;
import org.apache.poi.ss.usermodel.DataFormatter;
import org.apache.poi.ss.usermodel.FormulaEvaluator;
import org.apache.poi.ss.usermodel.Row;
import org.apache.poi.ss.usermodel.Sheet;
import org.apache.poi.ss.usermodel.Workbook;
import org.apache.poi.ss.usermodel.WorkbookFactory;

public final class ExcelDataReader {
  private ExcelDataReader() { }

  public static Object[][] readSheet(String filePath, String sheetName,
                                     boolean skipHeader) throws IOException {
    List<Object[]> rows = new ArrayList<>();
    try (InputStream input = Files.newInputStream(Paths.get(filePath));
         Workbook workbook = WorkbookFactory.create(input)) {
      Sheet sheet = workbook.getSheet(sheetName);
      if (sheet == null) {
        throw new IllegalArgumentException("Sheet not found: " + sheetName + " in " + filePath);
      }
      Row firstRow = sheet.getRow(0);
      if (firstRow == null || firstRow.getLastCellNum() < 1) {
        throw new IllegalArgumentException("Sheet is empty or has no columns: " + sheetName);
      }
      int firstDataRow = skipHeader ? 1 : 0;
      int columnCount = firstRow.getLastCellNum();
      DataFormatter formatter = new DataFormatter();
      FormulaEvaluator evaluator = workbook.getCreationHelper().createFormulaEvaluator();
      for (int r = firstDataRow; r <= sheet.getLastRowNum(); r++) {
        Row row = sheet.getRow(r);
        if (row == null) continue;
        Object[] values = new Object[columnCount];
        boolean hasAnyValue = false;
        for (int c = 0; c < columnCount; c++) {
          Cell cell = row.getCell(c, Row.MissingCellPolicy.RETURN_BLANK_AS_NULL);
          String value = cell == null ? "" : formatter.formatCellValue(cell, evaluator).trim();
          values[c] = value;
          if (!value.isEmpty()) hasAnyValue = true;
        }
        if (hasAnyValue) rows.add(values);
      }
    }
    if (rows.isEmpty()) {
      throw new IllegalArgumentException("No test data rows found in " + filePath);
    }
    return rows.toArray(new Object[0][]);
  }
}
```

What the safeguards do

Code / concept	Why it matters
WorkbookFactory.create(input)	Can open supported Excel formats without hard-coding XSSFWorkbook.
try-with-resources	Closes the file stream and workbook reliably.
sheet == null check	Reports a useful error when the tab name is wrong.
DataFormatter	Converts text/numeric/date/formula cells into strings based on their displayed values.
FormulaEvaluator	Evaluates formulas when formatting formula cells.
MissingCellPolicy.RETURN_BLANK_AS_NULL	Lets the code handle missing cells instead of dereferencing null.
hasAnyValue	Skips completely blank rows so they do not become empty test invocations.
rows.toArray(new Object[0][])	Converts the collected data rows into the Object[][] return type expected by this DataProvider use case.

Design choice

This reader returns Strings for simplicity. If your tests need true numbers, booleans, dates, or complex objects, build a typed conversion layer rather than parsing every cell as a String inside the test method.

9. Connect the Excel reader to TestNG

DataProvider class reads Excel data

```
package providers;

import data.ExcelDataReader;
import java.io.IOException;
import org.testng.annotations.DataProvider;

public class RegisterDataProvider {
   @DataProvider(name = "registerTestData")
   public static Object[][] registerTestData() throws IOException {
      return ExcelDataReader.readSheet(
         "src/test/resources/testdata/Register.xlsx",
         "Register",
         true   // skip the header row
      );
   }
}
```

Use the provider from a test class

TestNG method uses the external provider

```
import org.testng.annotations.Test;
import org.testng.Assert;

public class RegisterDataMappingTest {
   @Test(dataProvider = "registerTestData",
         dataProviderClass = RegisterDataProvider.class)
   public void registerTestCase(String firstName,
                                String lastName,
                                String phone) {
      Assert.assertNotNull(firstName, "First name must be present");
      Assert.assertNotNull(lastName, "Last name must be present");
      Assert.assertNotNull(phone, "Phone must be present");
      // Use the values in a UI/API test and assert the actual result.
   }
}
```

Because the provider is in another class, dataProviderClass identifies it. The provider method is static in this example; TestNG can then call it without needing to construct the provider class.

Excel template

FirstName	LastName	Phone
Amol	Jadhav	9090901001
Rutuja	Kale	9090901002
Sonali	Jadhav	9090901003
Vikas	Mule	9090901004

Store the first row as column headers. Because skipHeader=true, the data provider starts from the second spreadsheet row. The three remaining columns are mapped to the three String parameters in the test method.

10. Complete Selenium + TestNG example

This illustrates the pattern in your notes: open the registration page, enter first name, last name and phone from Excel, then assert the values were entered. The supplied source uses the Guru99 New Tours demo URL; it is a legacy training site and may change, so confirm that the page and locators still match before using the example in a live project.

Illustrative UI test using the Excel-backed DataProvider

```
package tests;

import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
import org.testng.annotations.DataProvider;

public class RegisterTest {
   private WebDriver driver;
   private WebDriverWait wait;

   @BeforeMethod
   public void setUp() {
      driver = new ChromeDriver();
      driver.manage().window().maximize();
      wait = new WebDriverWait(driver, Duration.ofSeconds(10));
      String url = System.getProperty(
         "app.url",
         "https://demo.guru99.com/test/newtours/register.php");
      driver.get(url);
   }

   @DataProvider(name = "registerTestData")
   public Object[][] registerTestData() throws java.io.IOException {
      return data.ExcelDataReader.readSheet(
         "src/test/resources/testdata/Register.xlsx",
         "Register",
         true);
   }

   @Test(dataProvider = "registerTestData")
   public void registerTest(String firstName,
                            String lastName,
                            String phone) {
      WebElement first = wait.until(ExpectedConditions
         .visibilityOfElementLocated(By.name("firstName")));
      WebElement last = driver.findElement(By.name("lastName"));
      WebElement phoneField = driver.findElement(By.name("phone"));

      first.sendKeys(firstName);
      last.sendKeys(lastName);
      phoneField.sendKeys(phone);

      Assert.assertEquals(first.getAttribute("value"), firstName);
      Assert.assertEquals(last.getAttribute("value"), lastName);
      Assert.assertEquals(phoneField.getAttribute("value"), phone);
   }

   @AfterMethod(alwaysRun = true)
   public void tearDown() {
      if (driver != null) {
         driver.quit();
      }
   }
}
```

Important explanation

- @BeforeMethod runs before each data-driven invocation, so each row gets a fresh browser in this example.
- @AfterMethod(alwaysRun = true) closes the browser even if the test fails.
- WebDriverWait waits for the first field to be visible; fixed Thread.sleep calls are avoided.
- Assertions verify field values rather than merely proving that sendKeys did not throw an exception.
- The sample verifies entered values only. A real registration test should submit the form and assert a trustworthy success/error message or a backend result.
- If the test is run in parallel, each invocation must have an independent WebDriver instance and isolated test data.

11. Real-time scenarios and data design

Scenario A: registration form

Row category	Example input	Expected outcome
Positive	Valid first name, last name, phone	Form accepts values and completes registration.
Boundary	Maximum supported name length	Accepted up to the requirement limit; longer input handled correctly.
Negative	Empty first name	Validation message appears and registration is blocked.
Negative	Alphabetic characters in phone field	Validation follows the specified rule.
Duplicate	Already-used phone/account	Application prevents or handles duplicates according to requirements.

Scenario B: login tests

Username	Password	Expected result
valid.user	valid password	Dashboard or home page appears.
valid.user	wrong password	Authentication error shown.
locked.user	any password	Locked-account message shown.
blank	blank	Required-field messages shown.

Scenario C: API testing

The same DataProvider pattern works for REST Assured. Each row can include username, role, expected HTTP status, and expected response message. Keep expected results in the dataset only when it makes the test clearer; avoid turning the spreadsheet into a complicated programming language.

Conceptual API example

```
@DataProvider(name = "apiCases")
public Object[][] apiCases() {
   return new Object[][] {
      { "known-user", 200, "ACTIVE" },
      { "unknown-user", 404, "NOT_FOUND" }
   };
}

@Test(dataProvider = "apiCases")
public void getUserStatus(String userId,
                          int expectedStatus,
                          String expectedState) {
   // Send request with REST Assured.
   // Assert HTTP status and response state against expected values.
}
```

12. Troubleshooting common errors

Symptom	Likely cause	Fix
FileNotFoundException	Wrong absolute/relative path; file not copied into test resources.	Use a stable project-relative path or classpath resource; check the filename and case.
Sheet is null	Spelling/case mismatch in getSheet().	Confirm the worksheet tab name in Excel.
NullPointerException	Missing row or cell, or a blank row.	Check for null rows/cells and skip or report invalid data.
IllegalStateException from getStringCellValue()	Cell contains a number, date, boolean, or another type.	Use DataFormatter or explicitly convert the cell type.
TestNG parameter mismatch	Row has a different count/order of values than test method parameters.	Match column order and parameter list; validate the workbook schema.
Test executes with header data	Header row was not skipped.	Set skipHeader=true or start the loop at row index 1.
Phone loses leading zero	Excel stored phone as a number.	Format the cell as Text and store the phone as a string.
Only some rows are loaded	Fixed array size is too small or manual assignments omit rows.	Build Object[][] dynamically from the actual sheet.
Workbook/file remains locked	Resources were not closed.	Use try-with-resources for InputStream and Workbook.
Flaky Selenium test	Fixed sleeps, shared state, unstable locators, or no assertions.	Use explicit waits, reliable locators, isolated browser instances, and meaningful assertions.

13. Best practices for industry-ready data-driven tests

Excel data design

- Use one clear header row and one scenario per data row.

- Keep column order and names stable; treat the sheet as a contract between QA data and test code.
- Use text format for phone numbers, postal codes, IDs, and any value where leading zeros matter.
- Keep expected results explicit for negative and boundary tests.
- Use meaningful filenames and worksheet names; validate required columns before execution.
- Keep only non-sensitive test data in version control. Do not store real passwords, access tokens, or personal customer information in an Excel file committed to Git.

Java and Apache POI

- Use try-with-resources to close InputStream and Workbook.
- Avoid developer-specific absolute paths; prefer src/test/resources or a configurable path.
- Use WorkbookFactory when the same reader needs to support multiple Excel formats.
- Use DataFormatter for display-oriented strings and FormulaEvaluator for formula results.
- Validate workbook existence, sheet name, header, row shape, and mandatory values before the browser opens.
- Use a separate reader utility and provider class to keep test methods focused on behavior rather than file parsing.

TestNG and Selenium

- Keep the @DataProvider name exactly aligned with the dataProvider attribute.
- Match method parameters to data columns in the correct order; use meaningful parameter names.
- Use one test invocation per row and assert the actual behavior or application response.
- Prefer explicit waits to Thread.sleep; avoid unnecessary hard-coded delay.
- Use @AfterMethod(alwaysRun = true) for driver cleanup. Do not share one WebDriver across parallel invocations.
- Only enable @DataProvider(parallel = true) after ensuring browser isolation, test data independence, and thread safety.
- For very large datasets, evaluate CSV/JSON/database sources and test-data setup APIs; Excel is convenient but is not always the best source.

14. Execution count, scope, and parallel data providers

If the Excel worksheet has four valid data rows and the test method is linked to that DataProvider, TestNG ordinarily invokes the method four times. If the provider returns zero rows, the test has no data invocations and should be treated as a data setup problem, not silently considered meaningful coverage.

Optional parallel provider

```
@DataProvider(name = "registerTestData", parallel = true)
public Object[][] registerTestData() throws IOException {
    return ExcelDataReader.readSheet(
        "src/test/resources/testdata/Register.xlsx",
        "Register",
        true);
}
```

Parallel execution caution

Parallel providers may improve speed but can expose races: shared WebDriver variables, shared account state, duplicate usernames, or tests overwriting the same record. Start with serial execution. Add parallel=true only when each row can safely execute independently.

15. Recommended end-to-end workflow

17. Create the Excel workbook with a header row and valid scenario rows.
18. Save it under src/test/resources/testdata/Register.xlsx or a configurable test-data folder.
19. Add the Apache POI poi-ooxml dependency compatible with your project JDK.
20. Implement one reusable reader that validates the sheet and formats cell values safely.
21. Expose the reader through a TestNG @DataProvider returning Object[][].
22. Bind the provider using @Test(dataProvider = "registerTestData").
23. Use each row in a UI/API test and assert the real expected outcome.
24. Close browser/session resources with TestNG teardown methods.
25. Run the suite and confirm the number of invocations matches the valid data rows.
26. When a case fails, report the row or scenario name in test output so the input can be identified quickly.

16. Hands-on exercises

27. Create a Register.xlsx sheet containing FirstName, LastName, and Phone columns with five data rows. Read and print the first row after the header.
28. Write a DataProvider returning four rows and three values in each row. Verify the test method executes four times.
29. Replace hard-coded values with an ExcelDataReader and skip the header row.
30. Add a blank row in the middle of the worksheet. Confirm that the reader skips it instead of launching an empty test.
31. Add an empty mandatory phone value. Decide whether to reject the dataset before running Selenium or to test the application's blank-phone validation. Explain the difference.
32. Extend the sheet with ExpectedOutcome. Use positive and negative cases and assert the correct behavior for each row.
33. Adapt the data-provider concept to an API test with columns: UserId, ExpectedStatus, ExpectedState.

Suggested answers / checkpoints

- Rows and columns are zero-based in POI; a header at Excel row 1 is row index 0.
- Four non-empty data rows after the header should produce four data-driven test invocations.
- The test method arguments should match the three columns and order: firstName, lastName, phone.
- A blank row should normally be ignored; a blank mandatory value inside a non-empty row should be caught by data validation or intentionally used as a negative test.
- Do not make every invalid row a test-data parsing error: distinguish malformed test data from intentionally invalid application input.

17. Interview questions and quick revision

What is data-driven testing? Executing the same test logic with multiple input datasets.

What does Apache POI do? Provides Java APIs to read and write Office formats such as Excel.

Which API is used for .xlsx? XSSF / XSSFWorkbook; poi-ooxml provides modern OOXML spreadsheet support.

What is @DataProvider? A TestNG annotation that marks a method as a data source for test-method invocations.

Why Object[][]? Each outer row provides one invocation, and each inner value becomes one test-method argument.

If Object[][] has 5 rows and 3 columns, how many invocations and parameters? Normally 5 invocations; 3 arguments per invocation.

What does getLastRowNum() return? The zero-based index of the last row known to the sheet, not simply the number of non-empty rows.

Why is getStringCellValue risky? It assumes the cell stores a string; numeric and other cell types can raise an exception.

Why use DataFormatter? It converts cells to display-oriented strings across several cell types and can work with formula evaluation.

Why must phone numbers be strings? They are identifiers; numeric storage may remove leading zeros or formatting.

Why skip the header row? Column labels must not be passed as input to the test method.

Why use try-with-resources? It ensures the stream and workbook are closed on success or failure.

Can @DataProvider run in parallel? Yes, but browser instances, test data, and shared state must be isolated and thread-safe.

Does filling fields alone prove registration succeeded? No. The test must assert a result such as confirmation text, navigation, or the appropriate API/backend state.

18. Official references

These references support the API concepts used in this guide. Check the current version documentation before changing dependencies or upgrading your framework.

- **Apache POI download / release notes:** https://poi.apache.org/download.cgi
- **Apache POI component overview:** https://poi.apache.org/components/index.html
- **TestNG documentation: DataProvider and test methods:** https://testng.org/documentation.html

Version context checked for this guide: the Apache POI download page reported POI 6.0.0 as the latest stable release on 22 October 2026. The POI 6.0.0 release announcement notes a minimum Java version of 17. If your project uses an earlier JDK or an existing managed dependency set, select the compatible POI release rather than upgrading blindly.

Final takeaway

Excel supplies the data; Apache POI reads it; TestNG @DataProvider maps each row to a test invocation; Selenium or an API client executes the scenario; assertions decide whether the behavior is correct. Keep these responsibilities separate for a maintainable automation framework.